

Universidad San Carlos de Guatemala

Centro universitario de Occidente

División de Ciencias de la ingeniería

Ingeniería en Ciencias y Sistemas

Estructura De Datos

Ing. Mario Ramirez Tobar



TADs
PRACTICA 1
JUEGO UNO

Cristian Alejandro Roldán López
Carné: 202147280

Quetzaltenango 24 de febrero de 2026

Carta

Creación de las cartas para poder utilizarlas en el juego con los distintos métodos y atributos

Modelo

Interfaz Pública

```
    Carta();  
Carta(ColorCarta color, TipoCarta tipo, ValorCarta  
      valor);  
Carta(const Carta& otra) : color(otra.color),  
    tipo(otra.tipo), valor(otra.valor) {}  
  
    ColorCarta getColor();  
void setColor(ColorCarta color);  
    TipoCarta getTipo();  
void setTipo(TipoCarta tipo);  
    ValorCarta getValor();  
void setValor(ValorCarta valor);  
  
    void mostrarCarta();  
  
vector<std::string> lineaHorizontal();
```

Interfaz Privada

```
ColorCarta color;  
TipoCarta tipo;  
ValorCarta valor;
```

CartaFlip

Creación de carta para jugar el modo Flip utilizando referencias de Carta

Modelo

Interfaz Pública

```
    CartaFlip();  
    CartaFlip(Carta claro, Carta oscuro);  
  
    Carta getCartaActual(bool ladoOscuro);  
    void mostrarCartaActual(bool ladoOscuro);  
        void mostrarAmbosLados();  
    void setCartaActual(Carta& carta, bool ladoOscuro);  
    vector<std::string> lineaHorizontalFlip();
```

Interfaz Privada

```
    Carta ladoClaro;  
    Carta ladoOscuro;
```

Jugador

Creación del jugador implementado las listas que servirán para los mazos que jugara

Modelo

Interfaz Pública

```
Jugador() ;
Jugador(string nombre) ;

void recibirCarta(Carta carta) ;
void recibirCartaFlip(const CartaFlip& cartaFlip) ;
void mostrarMano() ;
void mostrarManoConIndices() ;
int cantidadCartas() ;
int cantidadaCartaFlip() ;
Carta jugarCarta(int posicionCarta) ;
bool tieneUno() ;
bool ganoJugador() ;
bool ganoJugadorFlip() ;
string getNombreJugador() ;
void setNombreJugador(string nombreJugador) ;
CartaFlip jugarCartaFlip(int posicionCarta) ;

void mostrarManoFlipConIndices(bool modoOscuro)
const;
```

Interfaz Privada

```
ListaMazoJugador manoJugador;
ListaMazoFlipJugador mazoFlipJugador;
string nombreJugador;
bool gane = false;
```

TADs
LISTAS
MAZOS
JUGADOR Y
LISTA
CIRCULAR

ListaCircular

Lista circular doblemente enlazada para poder moverse entre los jugadores

Modelo

Interfaz Pública

```
ListaCircular() ;  
~ListaCircular() ;  
void agregarPrimero(Jugador j) ;  
void agregarUltimo(Jugador j) ;  
bool estaVacía() ;  
int cantidad() ;
```

Interfaz Privada

```
NodoCircular *raiz;
```

NodoMano

Modelo

Interfaz Pública

```
NodoMano () ;  
NodoMano (Carta)
```

Interfaz Privada

```
Carta carta ;  
NodoMano* siguiente ;
```

NodoManoFlip

Modelo

Interfaz Pública

```
NodoManoFlip () ;  
NodoManoFlip (const CartaFlip& carta) ;  
CartaFlip getCartaFlip () ;  
NodoManoFlip* getSiguiente () ;  
void setSiguiente (NodoManoFlip* sig) ;
```

Interfaz Privada

```
CartaFlip cartaFlip ;  
NodoManoFlip* siguiente ;
```

ListaMazoJugador

Modelo

Interfaz Pública

```
ListaMazoJugador() ;  
~ListaMazoJugador() ;  
  
void insertarCartasOrdenada(Carta carta) ;  
void mostrarMazoJugador() ;  
void mostrarMazoJugadorConIndices() ;  
int sizeMazoJugador() ;  
Carta jugarCarta(int posicionCarta) ;  
Carta obtenerCarta(int posicionCarta) ;  
void eliminarCarta(int posicionCarta) ;
```

Interfaz Privada

```
ListaMazoJugador() ;  
~ListaMazoJugador() ;  
  
void insertarCartasOrdenada(Carta carta) ;  
void mostrarMazoJugador() ;  
void mostrarMazoJugadorConIndices() ;  
int sizeMazoJugador() ;  
Carta jugarCarta(int posicionCarta) ;  
Carta obtenerCarta(int posicionCarta) ;  
void eliminarCarta(int posicionCarta) ;
```

ListaMazoJugadorFlip

Modelo

Interfaz Pública

```
ListaMazoFlipJugador() ;  
~ListaMazoFlipJugador() ;  
  
void insertarOrdenado(const CartaFlip& cartaFlip) ;  
CartaFlip obtenerCartaFlip(int posicion) ;  
int sizeMazoFlipJugador() ;  
void eliminarCartaFlip(int posicion) ;  
void mostrarConIndices(bool modoOscuro = false)  
    const ;
```

Interfaz Privada

```
NodoManoFlip* head ;  
int cantidad ;  
bool ladoOscuro ;  
bool ordenar(CartaFlip a, CartaFlip b) ;
```

TADs
LISTAS
CARTAS
NORMALES
Y FLIP Y
PILA DE
CARTAS

NodoMazo

Modelo

Interfaz Pública

```
NodoMazo();  
NodoMazo(Carta c): carta(c), sigueinte(nullptr){};  
NodoMazo(CartaFlip cartaFlip): cartaFlip(cartaFlip),  
    sigueinte(nullptr){};
```

Interfaz Privada

```
Carta carta;  
NodoMazo* sigueinte;  
CartaFlip cartaFlip;
```

NodoFlip

Modelo

Interfaz Pública

```
NodoFlip();  
NodoFlip(CartaFlip cartaFlip);  
  
CartaFlip getCartaFlip();  
NodoFlip* getSiguiente();  
  
void setSiguiente(NodoFlip* siguiente);
```

Interfaz Privada

```
CartaFlip cartaFlip;  
NodoFlip* siguiente;
```

MazoCartas

Modelo

Interfaz Pública

```
MazoCartas();  
MazoCartas(int cantidadJugadores);  
Carta robarCarta();  
int cartasRestantes();  
void revolverCartas();  
void mostrarMazoCompleto();
```

Interfaz Privada

```
PilaMazo mazo;  
void createMazoInicial();
```

MazoFlip

Modelo

Interfaz Pública

```
MazoFlip();  
MazoFlip(int cantidadJugadores);  
CartaFlip robarCartaFlip();  
int cantidadRestantes();  
bool isEmpty();  
void mostrarMazoFlip();  
void revolverCartaMazoFlip();
```

Interfaz Privada

```
PilaFlip mazoFlip;  
void createMazoFlip();
```

PilaMazo

Modelo

Interfaz Pública

```
PilaMazo();  
void push(Carta carta);  
Carta pop();  
Carta peek();  
bool esVacia();  
int sizeMazo();
```

Interfaz Privada

```
NodoMazo* cabeza;  
int cantidad;
```

PilaFlip

Modelo

Interfaz Pública

```
PilaFlip();  
void push(CartaFlip cartaFlip);  
CartaFlip pop();  
CartaFlip peek();  
bool isEmpty();  
int sizeMazoFlip();
```

Interfaz Privada

```
NodoFlip* head;  
int cantidad;
```

ConfiguracionPartida

Modelo

Interfaz Pública

```
ConfiguracionPartida() ;  
    void mostrarMenu() ;  
    void configurar() ;  
    bool isAcumulacionActiva() ;  
    bool isRetoMas4Activo() ;  
bool isRobarHastaEncontrar() ;  
    bool isGritoUnoActivo() ;  
    bool isGanarConNegra() ;
```

Interfaz Privada

```
    bool acumulacionCartas ;  
    bool retoMas4Activo ;  
bool robarHastaEncontrar ;  
    bool gritoUnoActivo ;  
    bool ganarConNegra ;
```

TADs

LÓGICA DE

JUEGO

CrearJugador

Modelo

Interfaz Pública

```
CrearJugador() ;  
int solicitarJugadores() ;  
Jugador* crearJugadores(int& cantidad) ;
```

Interfaz Privada

```
int cantidadJugadores ;
```

EfectoCarta
Modelo
Interfaz Pública

```
    EfectoCartas();  
    EfectoCartas(ConfiguracionPartida* config);  
    bool cartaValida(Carta& cartaJugada, Carta&  
        cartaMesa);  
    bool efectoCarta(Carta& carta, TurnosJuego& turnos,  
        MazoCartas& mazo, Jugador* jugadores, int  
            cantidadJugadores);  
    bool victoriaJugador(Jugador& jugador);  
    bool cartaValidaFlip(CartaFlip& cartaJugadaFlip,  
        CartaFlip& cartaMesaFlip, bool ladoOscuro);  
    bool efectoCartaFlip(  
        CartaFlip& carta,  
        TurnosJuego& turnos,  
        MazoFlip& mazoFlip,  
        Jugador* jugadores,  
        int cantidadJugadores,  
        bool& ladoOscuro  
    );  
    ColorCarta pedirColor();  
    void robarHastaColor(Jugador& jugador, MazoFlip&  
        mazo, ColorCarta colorBuscado, bool ladoOscuro);
```

Interfaz Privada

```
    ConfiguracionPartida* configuracion;  
    bool ladoOscuro;
```

TurnosJuego

Modelo

Interfaz Pública

```
TurnosJuego();  
TurnosJuego(Jugador* jugador, int cantidadJugadores);  
Jugador& jugadorActual();  
void saltarTurno();  
void siguienteTurno();  
void invertirDireccion();  
Jugador& getJugador(int indice);  
int getIndiceActual();
```

Interfaz Privada

```
Jugador* jugador;  
int cantidadJugadores;  
int indiceActual;  
int direccion;
```

GritoUno

Modelo

Interfaz Pública

```
GritoUNO();  
void verificarSiDebeGritar(TurnosJuego& turnos);  
void intentarReporte(TurnosJuego& turnos, MazoCartas&  
                    mazo);  
void limpiarEstado();
```

Interfaz Privada

```
bool unoPendiente;  
int indiceJugadorPendiente;
```

Juego Modelo

Interfaz Pública

```
Juego() ;  
~Juego() ;  
void iniciarJuego() ;  
void loopJuego() ;  
void reutilizarPilaDescarte() ;  
void reutilizarPilaFlip() ;  
void limpiarPantalla() ;
```

Interfaz Privada

```
ConfiguracionPartida configuracion;  
Jugador* jugadores;  
int cantidadJugadores;  
MazoCartas mazo;  
MazoFlip mazoFlip;  
TurnosJuego turnosJuego;  
EfectoCartas efectoCartas;  
GritoUNO gritoUno;  
Carta cartaMesa;  
CartaFlip cartaMesaFlip;  
bool modoFlip;  
bool ladoOscuro;  
bool ganarJoker = false;  
int indiceGanadorJoker = -1;  
vector<Carta> pilaDescarte;  
vector<CartaFlip> pilaDescarteFlip;  
void loopNormal() ;  
void loopFlip() ;  
bool cartaValidaFlip(CartaFlip jugada, CartaFlip  
mesa) ;  
void victoriaPorJoker(int indiceJugador) ;
```

Nodo Mazo

ReparteCartasJugador

Modelo

Interfaz Pública

```
ReparteCartasJugador();  
void repartirCartasJugador(MazoCartas& mazo, Jugador*  
    jugador, int cantidadJugadores);  
void repartirCartasFlip(MazoFlip& mazoFlip, Jugador*  
    jugador, int cantidadJugadres);
```

TADs ENUM

ColorCarta

```
enum ColorCarta {  
    ROJO,  
    VERDE,  
    AMARILLO,  
    AZUL,  
    ROSA,  
    TURQUESA,  
    NARANJA,  
    VIOLETA,  
    MULTICOLOR,  
    NEGRA  
};
```

TipoCarta

```
enum TipoCarta {  
    CARTA_NUMERO,  
    CARTA_COMODIN,  
    CARTA_ACCION,  
    JOKER,  
    GLOTON  
};
```

ValorCarta

```
enum ValorCarta {  
    CERO,  
    UNO,  
    DOS,  
    TRES,  
    CUATRO,  
    CINCO,  
    SEIS,
```

*SIETE,
OCHO,
NUEVE,*

*TOMA_1,
TOMA_2,
TOMA_3,
TOMA_4,
TOMA_6,*

*SALTO,
SALTO_TODOS,
REVERSE,
CAMBIA_COLOR,
COLOR_ETERNO,
FLIP,
VALOR_JOKER,
VALOR_GLOTON
};*